

**ILLINOIS INSTITUTE OF TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE**

Database Organization

Course CS-425 02

Homework No.5

SQL Advanced window functions & OLAP queries

Professor:

Gerald Balekaki, Ph.D.

T.A.:

Manoj Kumar Yepuri

Names:

Jisun Yun

Daniel Kanjuni

Harlee Ramos

Due Date:

07/12/2024

Summer 2024

Answer the following database queries:

1. Output the number of rows in each of your 4 relations (using 4 SELECT statements) in this order: Coach, Person, Player, Team. Call the result column LOADED each time.

SELECT COUNT(*) AS LOADED FROM Coach;

The screenshot shows a database query tool interface. At the top, there is a toolbar with various icons and a text box that says "Limit to 50000 rows". Below the toolbar, the query editor shows the following SQL statements:

```
1 • USE Homework5DB;
2
3 #1
4 • SELECT COUNT(*) AS LOADED FROM Coach;
5
6
7
```

The query results are displayed in a table with the following structure:

LOADED
33

SELECT COUNT(*) AS LOADED FROM Person;

The screenshot shows a database query tool interface. At the top, there is a toolbar with various icons and a text box that says "Limit to 50000 rows". Below the toolbar, the query editor shows the following SQL statements:

```
1 • USE Homework5DB;
2
3 #1
4 • SELECT COUNT(*) AS LOADED FROM Person;
5
6
7
```

The query results are displayed in a table with the following structure:

LOADED
5061

SELECT COUNT(*) AS LOADED FROM Player;

The screenshot shows a SQL IDE interface. At the top, there is a toolbar with icons for file operations, execution, and search. A status bar on the right indicates "Limit to 50000 rows". The query editor contains the following SQL code:

```
1 • USE Homework5DB;  
2  
3 #1  
4 • SELECT COUNT(*) AS LOADED FROM Player;  
5  
6  
7
```

Below the query editor, a progress bar shows 100% completion. The "Result Grid" tab is active, displaying the query results in a table:

LOADED
454

SELECT COUNT(*) AS LOADED FROM Team;

The screenshot shows a SQL IDE interface. At the top, there is a toolbar with icons for file operations, execution, and search. A status bar on the right indicates "Limit to 50000 rows". The query editor contains the following SQL code:

```
1 • USE Homework5DB;  
2  
3 #1  
4 • SELECT COUNT(*) AS LOADED FROM Team;  
5  
6  
7
```

Below the query editor, a progress bar shows 100% completion. The "Result Grid" tab is active, displaying the query results in a table:

LOADED
30

2. Output everything in the Team relation, in ascending (increasing) order of TmID.

```
SELECT *
FROM Team
ORDER BY TmID ASC;
```

[illegible]

3. Show the TmID of teams with Milwaukee in their name, but your query must cater for the fact that people spell this incorrectly – everyone starts with “MIL”, then somewhere later they have a “W” and even later a “K”. Example misspellings are Millwaukee, Milwakee, Milwuakee, Milwaukey, etc.

```
SELECT TmID
FROM Team
WHERE Name LIKE '%MIL%' AND
Name LIKE '%W%' AND
Name LIKE '%K%';
```

The screenshot shows a SQL query editor with the following query:

```
2 #3
3 • SELECT TmID
4 FROM Team
5 WHERE Name LIKE '%MIL%' AND
6 Name LIKE '%W%' AND
7 Name LIKE '%K%';
8
9
10
11
12
```

Below the query editor is a toolbar with a 'Result Grid' button, a 'Filter Rows' button, a search input, and an 'Edit' button. The result grid shows the following data:

TmID
MIL
HULL

4. What are the lowest and the highest number of Games (played) in the coach data? Call the first result column LOWEST and the second result column HIGHEST.

```
SELECT MIN(Games) AS LOWEST,
       MAX(Games) AS HIGHEST
FROM Coach;
```

The screenshot shows a SQL query editor with the following query:

```
USE Homework5DB;
#4
SELECT MIN(Games) AS LOWEST,
       MAX(Games) AS HIGHEST
FROM Coach;
```

Below the query editor is a toolbar with a 'Result Grid' button, a 'Filter Rows' button, a search input, and an 'Export' button. The result grid shows the following data:

LOWEST	HIGHEST
28	82

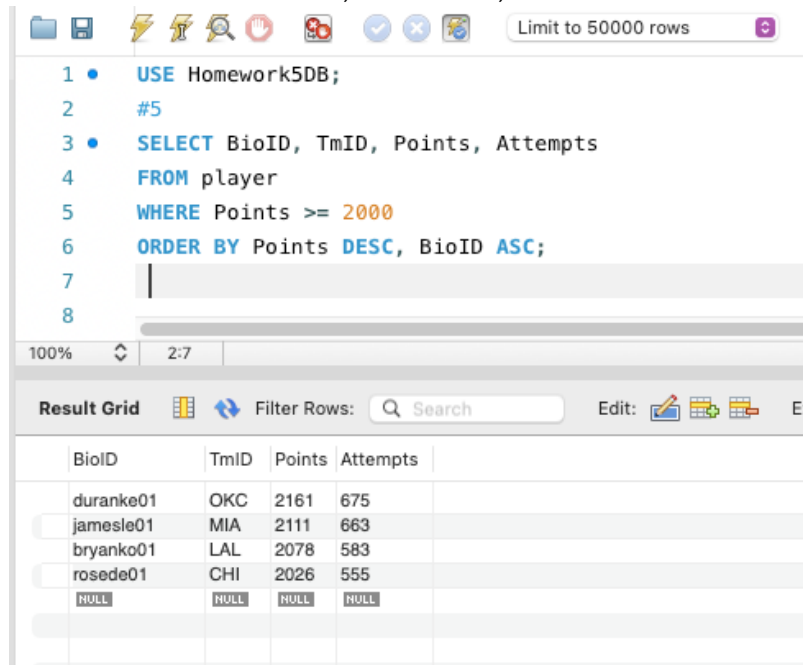
5. Show the BioID, TmID, Points and Attempts for each player that scored more than 2000 points, in decreasing order of Points. Players with the same number of Points should be shown in alphabetical order of BioID.

```
SELECT BioID, TmID, Points, Attempts
```

```
FROM player
```

```
WHERE Points >= 2000
```

```
ORDER BY Points DESC, BioID ASC;
```



The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
1 • USE Homework5DB;
2 #5
3 • SELECT BioID, TmID, Points, Attempts
4 FROM player
5 WHERE Points >= 2000
6 ORDER BY Points DESC, BioID ASC;
7
8
```

The results grid shows the following data:

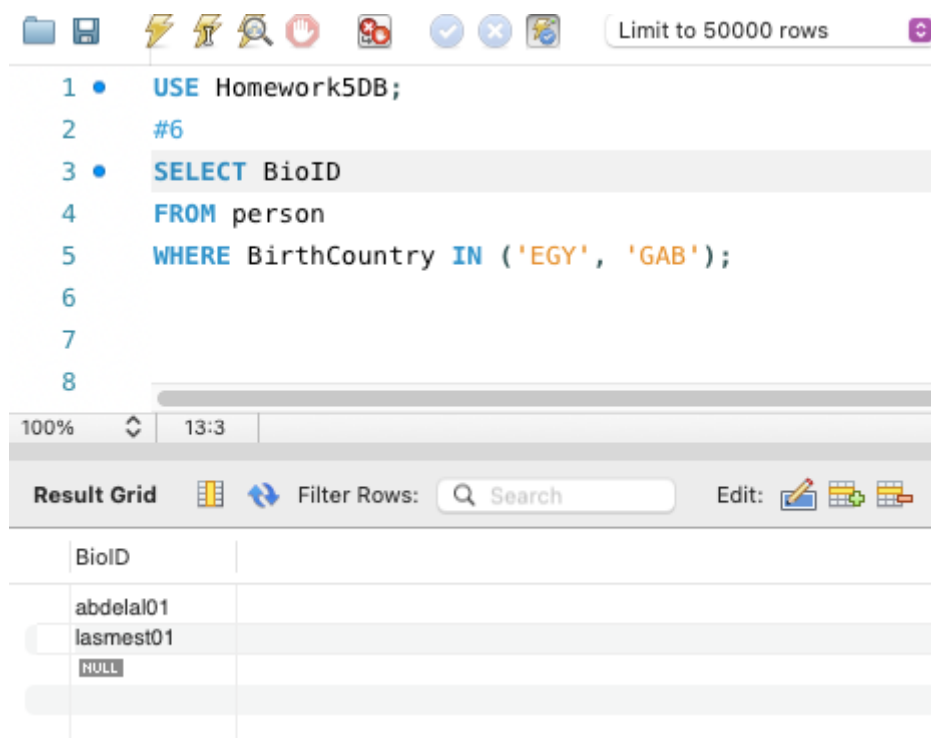
BioID	TmID	Points	Attempts
duranke01	OKC	2161	675
jamesle01	MIA	2111	663
bryanko01	LAL	2078	583
rosede01	CHI	2026	555
NULL	NULL	NULL	NULL

6. Show BioID of persons born in Gabon (GAB) and BioID of persons born in Egypt (EGY), if any.

```
SELECT BioID
```

```
FROM person
```

```
WHERE BirthCountry IN ('EGY', 'GAB');
```



The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
1 • USE Homework5DB;
2 #6
3 • SELECT BioID
4 FROM person
5 WHERE BirthCountry IN ('EGY', 'GAB');
6
7
8
```

The results grid shows the following data:

BioID
abdelal01
lasmest01
NULL

7. Show the following for Jamesle01: his Points, the full Name of his team, & the BioID of his team's coach.

```
SELECT p.Points, t.Name, c.BioID
FROM Player p
JOIN Team t ON p.TmID = t.TmID
JOIN Coach c ON t.TmID = c.TmID
WHERE p.BioID = 'jamesle01';
```

1 • **USE** Homework5DB;
 2 #7
 3 • **SELECT** p.Points, t.Name, c.BioID
 4 **FROM** Player p
 5 **JOIN** Team t **ON** p.TmID = t.TmID
 6 **JOIN** Coach c **ON** t.TmID = c.TmID
 7 **WHERE** p.BioID = 'jamesle01';
 8
 9

100% 1:8

Result Grid Filter Rows: Search Export:

Points	Name	BioID
2111	Miami Heat	spoeler01

8. Show the BioID of players whose BirthCountry is in the data, but their BirthCity is not in the data.
 SELECT *

FROM person

WHERE LENGTH(BirthCity) = 0 AND LENGTH(BirthCountry) > 0;

Limit to 50000 rows

1 • **USE** Homework5DB;
 2 #8
 3 • **SELECT** *
 4 **FROM** person
 5 **WHERE** LENGTH(BirthCity) = 0 **AND** LENGTH(BirthCountry) > 0;
 6
 7

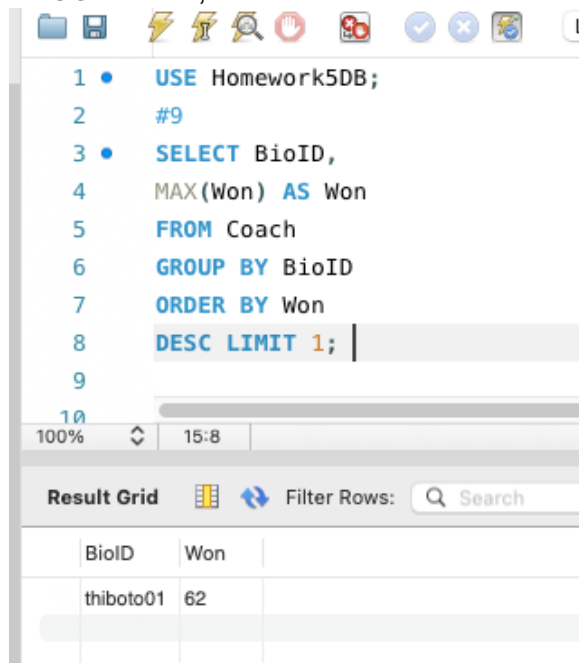
100% 1:8

Result Grid Filter Rows: Search Edit: Export/Import:

BioID	FirstName	LastName	BirthDate	BirthCity	BirthCountry
alvaral01	Armando	Alvarez	10/5/1912		ARG
ardonja01	Jaak	Ardon	10/13/1939		EST
bloedch01	Charles	Bloedorn	5/28/1912		USA
bowlior01	Orbie	Bowling	3/21/1939		USA
carlich01	Chester	Carlisle	11/2/1916		USA
carlsdo01	Donald	Carlson	3/22/1919		USA
charlke01	Kenneth	Charles	7/10/1951		TRI
croftbo01	Robert	Croft	8/10/1947		CAN
freyfr01	Frido	Frey	10/26/1921		GER
grimswo01	George	Grimshaw	9/24/1919		ENG
isaacjo01	John	Isaacs	9/15/1915		PAN
minorda01	Davage	Minor	2/23/1922		USA
robiner01	Ermer	Robinson	11/24/1922		USA
vocega01	Gary	Voce	11/24/1965		JAM
zellega01	Gary	Zeller	11/20/1947		USA
NULL	NULL	NULL	NULL	NULL	NULL

9. Which coach(es) Won the most games? Give BioID and the number Won.

```
SELECT BioID,  
MAX(Won) AS Won  
FROM Coach  
GROUP BY BioID  
ORDER BY Won  
DESC LIMIT 1;
```



The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

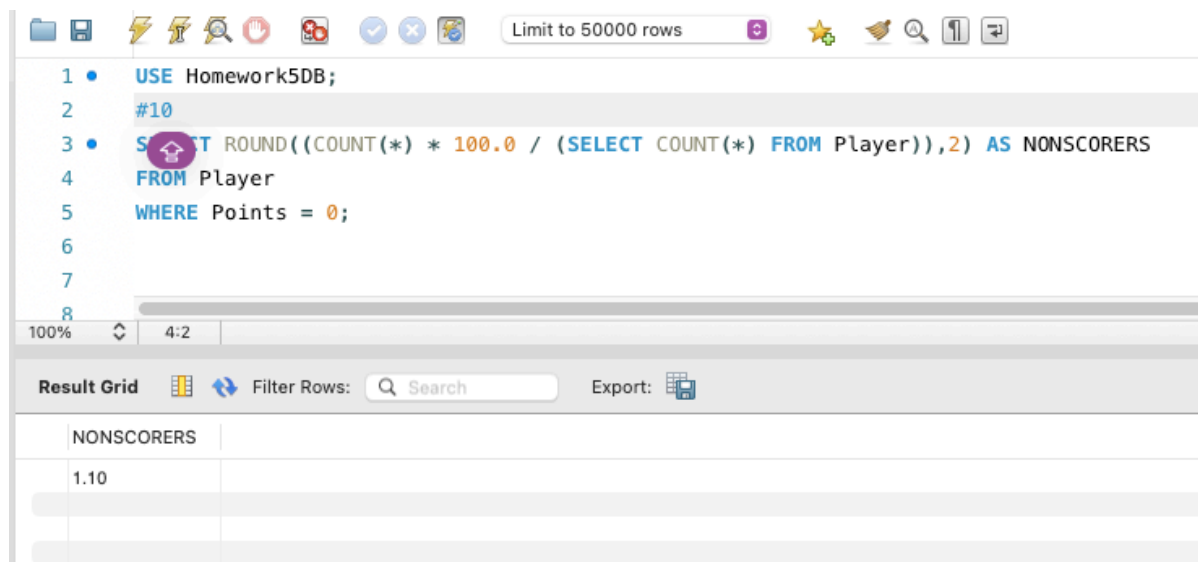
```
1 • USE Homework5DB;  
2 #9  
3 • SELECT BioID,  
4 MAX(Won) AS Won  
5 FROM Coach  
6 GROUP BY BioID  
7 ORDER BY Won  
8 DESC LIMIT 1;  
9  
10
```

Below the query editor, the 'Result Grid' is displayed with the following data:

BioID	Won
thiboto01	62

10. What percentage of players have Points scored as zero? Call the result column NONSCORERS.

```
SELECT ROUND((COUNT(*) * 100.0 / (SELECT COUNT(*) FROM Player)),2) AS NONSCORERS  
FROM Player  
WHERE Points = 0;
```



The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
1 • USE Homework5DB;  
2 #10  
3 • SELECT ROUND((COUNT(*) * 100.0 / (SELECT COUNT(*) FROM Player)),2) AS NONSCORERS  
4 FROM Player  
5 WHERE Points = 0;  
6  
7  
8
```

Below the query editor, the 'Result Grid' is displayed with the following data:

NONSCORERS
1.10

11. How many teams have Lost more games than they have Won? Call the result column LOSERS.
SELECT COUNT(*) AS LOSERS
FROM Team
WHERE Lost > Won;

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
1 • USE Homework5DB;  
2  
3 #11  
4 • SELECT COUNT(*) AS LOSERS  
5 FROM Team  
6 WHERE Lost > Won;  
7  
8
```

Below the query editor, the 'Result Grid' is displayed, showing the results of the query:

LOSERS
14

12. How many teams belong to each ConfID? Call the 2nd column CONFSIZE.

SELECT DISTINCT ConfID, COUNT(*) OVER (PARTITION BY ConfID) AS CONFSIZE
FROM team;

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
1 • USE Homework5DB;  
2  
3 #12  
4 • SELECT DISTINCT ConfID, COUNT(*) OVER (PARTITION BY ConfID) AS CONFSIZE  
5 FROM team;  
6  
7
```

Below the query editor, the 'Result Grid' is displayed, showing the results of the query:

ConfID	CONFSIZE
EC	15
WC	15

13. How many countries do the persons in this data come from? Call the result column NUMLANDS.

```
SELECT COUNT(DISTINCT BirthCountry) AS NUMLANDS
FROM Person
WHERE LENGTH(BirthCountry) > 0;
```

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
1 • USE Homework5DB;
2
3 #13
4 • SELECT COUNT(DISTINCT BirthCountry) AS NUMLANDS
5 FROM Person
6 WHERE LENGTH(BirthCountry) > 0;
7
8
```

The interface includes a toolbar at the top with icons for file operations, a "Limit to 50000 rows" dropdown, and a search icon. Below the query editor, the "Result Grid" is displayed, showing a single row with the value 80 under the column header NUMLANDS.

NUMLANDS
80

14. Which pairs of teams have the exact same record (meaning the same values for Won and the same values for Lost)? Show their Won value and then their Lost value and then the 2 Names, making sure that the 3rd column Name is alphabetically before the 4th column Name so information is not repeated. Call the 3rd column TEAM1 and the 4th column TEAM2.

```
SELECT t1.Won, t1.Lost, t1.Name AS TEAM1, t2.Name AS TEAM2
FROM Team t1
JOIN Team t2 ON t1.Won = t2.Won AND t1.Lost = t2.Lost AND t1.TmID < t2.TmID
ORDER BY TEAM1, TEAM2;
```

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

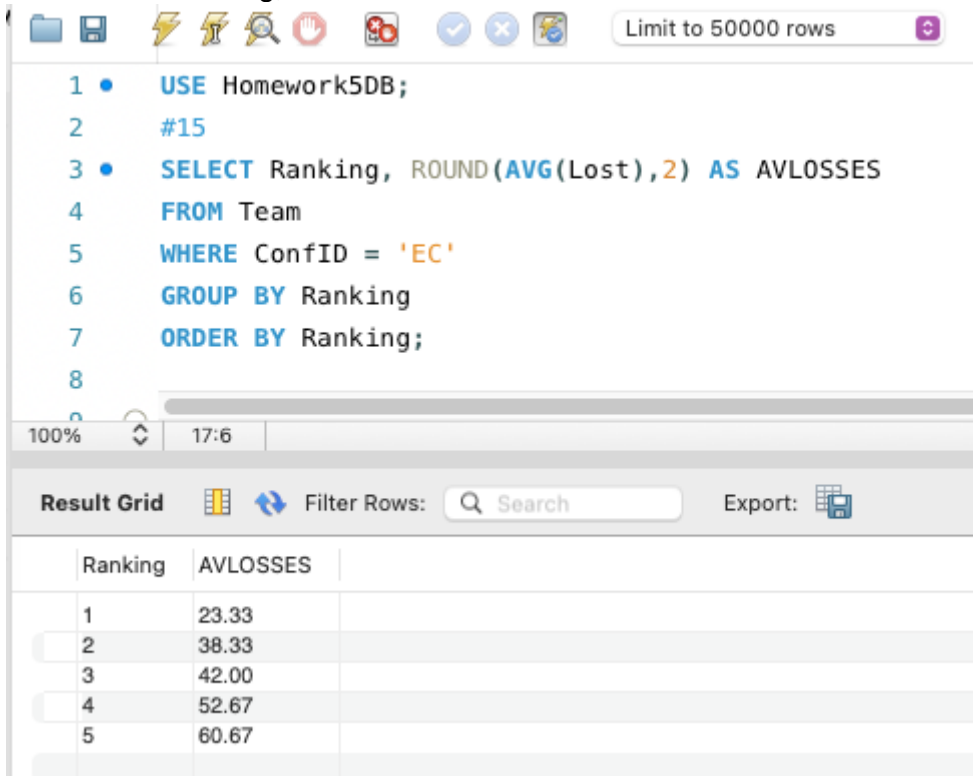
```
1 • USE Homework5DB;
2
3 #14
4 • SELECT t1.Won, t1.Lost, t1.Name AS TEAM1, t2.Name AS TEAM2
5 FROM Team t1
6 JOIN Team t2 ON t1.Won = t2.Won AND t1.Lost = t2.Lost AND t1.TmID < t2.TmID
7 ORDER BY TEAM1, TEAM2;
```

The interface includes a toolbar at the top with icons for file operations, a "Limit to 50000 rows" dropdown, and a search icon. Below the query editor, the "Result Grid" is displayed, showing a table with columns Won, Lost, TEAM1, and TEAM2. The results show three pairs of teams with identical win/loss records.

Won	Lost	TEAM1	TEAM2
57	25	Dallas Mavericks	Los Angeles Lakers
46	36	Memphis Grizzlies	New Orleans Hornets
24	58	New Jersey Nets	Sacramento Kings

15. For each of the 5 Rankings in ConfID "EC", show the average number of games Lost by those teams. Call the 2nd column AVLOSSES.

```
SELECT Ranking, ROUND(AVG(Lost),2) AS AVLOSSES
FROM Team
WHERE ConfID = 'EC'
GROUP BY Ranking
ORDER BY Ranking;
```



Limit to 50000 rows

```
1 • USE Homework5DB;
2 #15
3 • SELECT Ranking, ROUND(AVG(Lost),2) AS AVLOSSES
4 FROM Team
5 WHERE ConfID = 'EC'
6 GROUP BY Ranking
7 ORDER BY Ranking;
8
```

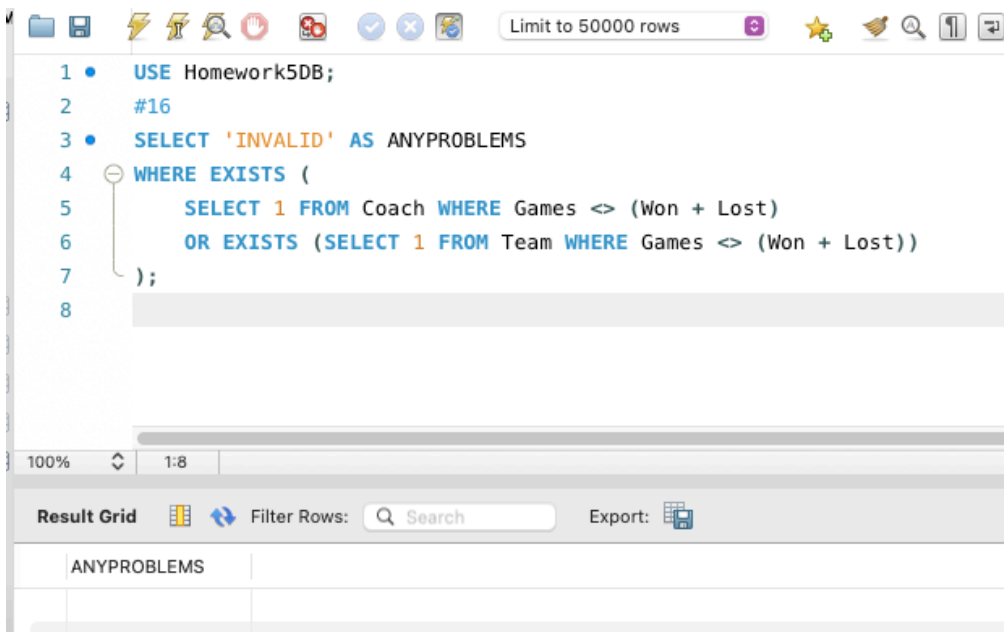
100% 17:6

Result Grid Filter Rows: Search Export:

Ranking	AVLOSSES
1	23.33
2	38.33
3	42.00
4	52.67
5	60.67

16. Give a SQL statement to output "INVALID" if any information in any row (tuple) of any relation has invalid data (e.g., the Games value is not equal to Won value plus Lost value). If all the data is valid then it should output an empty table. Call the result column ANYPROBLEMS.

```
SELECT 'INVALID' AS ANYPROBLEMS
WHERE EXISTS (
    SELECT 1 FROM Coach WHERE Games <> (Won + Lost)
    OR EXISTS (SELECT 1 FROM Team WHERE Games <> (Won + Lost)));
```



Limit to 50000 rows

```
1 • USE Homework5DB;
2 #16
3 • SELECT 'INVALID' AS ANYPROBLEMS
4 WHERE EXISTS (
5     SELECT 1 FROM Coach WHERE Games <> (Won + Lost)
6     OR EXISTS (SELECT 1 FROM Team WHERE Games <> (Won + Lost))
7 );
8
```

100% 1:8

Result Grid Filter Rows: Search Export:

ANYPROBLEMS
